Multi Agent Architecture

- Custonomy Agent Execution Architecture
  - 1. Objective
- 2. Core execution principle
  - Process once, decide many times
- 3. Recommended logical layers
  - Layer 1: Source data layer
  - Layer 2: Shared feature computation layer
    - Customer feature examples
    - Business feature examples
  - Layer 3: Signal layer
    - Example signals
  - Layer 4: Trigger and eligibility engine
    - Example triggers
  - Layer 5: Supervisor agent layer
  - Layer 6: Child agent layer
  - Layer 7: Next Best Action layer
  - Layer 8: Human approval and execution
- 4. Supervisor and child execution design
  - A. Revenue Recovery Supervisor
    - Purpose
    - Routing logic
    - Child agents
    - Compute rule
  - B. Customer Value Growth Supervisor
    - Purpose
    - Routing logic
    - Child agents
    - Compute rule
  - C. Campaign Intelligence Supervisor
    - Purpose
    - Routing logic
    - Child agents
    - Compute rule
  - D. Customer Experience Guardian Supervisor
    - Purpose
    - Routing logic
    - Child agents
    - Compute rule
  - E. Revenue Intelligence Supervisor
    - Purpose
    - Routing logic
    - Child agents

Compute rule
F. Seasonal Demand Intelligence Supervisor
Purpose
Routing logic
Child agents
Compute rule
G. Master Orchestrator
Next Best Action Agent
Purpose
Input sources
Compute rule
5. Processing strategy by compute cost
Tier 1: Cheap rules
Tier 2: Medium-cost scoring
Tier 3: Expensive decisioning / reasoning
6. Invocation policy
General rules
7. Processing cadence
Real-time / near real-time
Daily
Weekly
Monthly
8. Shared feature store design
Customer feature store
Campaign feature store
Segment / category feature store
9. Event-driven optimization model
Recommended event types
10. Example execution flow
Example 1: Revenue recovery
Example 2: Campaign planning
Example 3: Experience recovery
11. Compute optimization rules for design doc
Compute Optimization Principles
12. Recommended architecture summary table
13. Final recommended operating model
Best-fit Custonomy model

Custonomy Agent Execution Architecture

1. Objective

Design Custonomy so that:

- data is processed once and reused many times
- agents do not independently scan the same customer/order/event data

- only relevant agents are invoked
- compute cost stays controlled as data volume grows
- recommendations remain coordinated and non-conflicting

---

2. Core execution principle

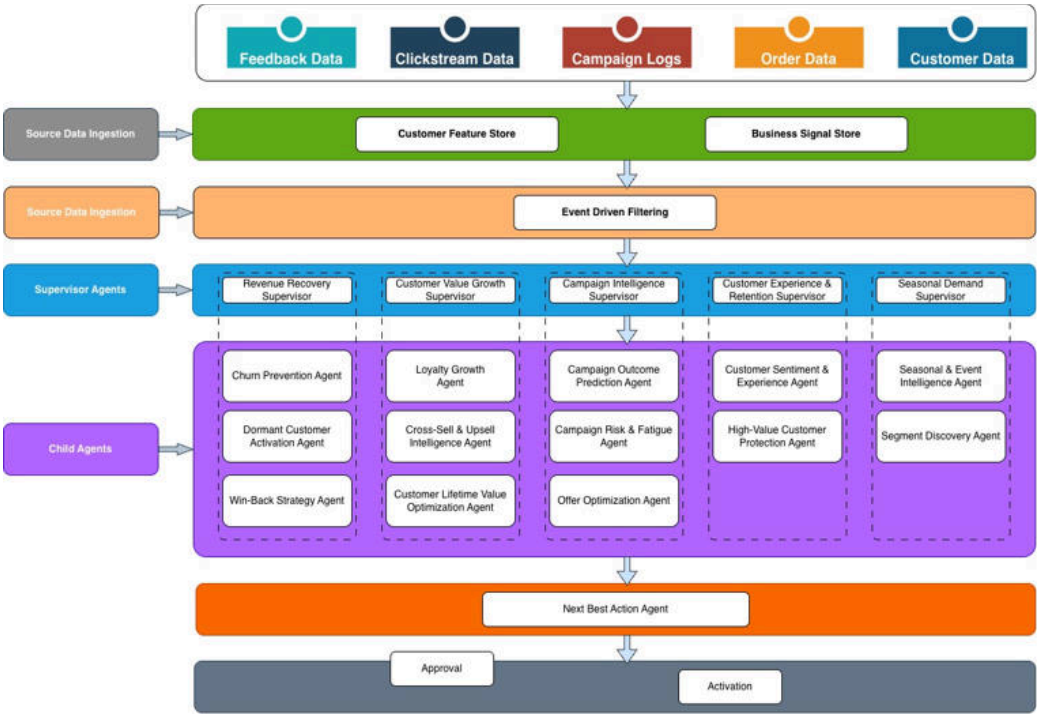
Process once, decide many times

Custonomy should not allow every agent to directly scan:

- customer master
- orders
- campaign logs
- browsing events
- support tickets
- delivery/return events

Instead, Custonomy should use:

- a **shared feature layer**
- a **shared signal layer**
- a **trigger engine**
- **supervisor agents** for routing
- **child agents** only for shortlisted populations
- a final **Next Best Action** orchestration layer



Agents Execution Process

---

3. Recommended logical layers

Layer 1: Source data layer

Raw enterprise data enters the platform from:

- Customer Data
- order transactions
- browsing / app / web engagement
- campaign execution logs
- loyalty data
- support / complaint / ticket systems
- product / category / pricing systems
- seasonal / event calendars

Layer 2: Shared feature computation layer

This layer computes reusable features once.

Customer feature examples

- last purchase date
- purchase frequency
- average basket size
- spend velocity
- spend decline %
- LTV
- churn score
- dormant bucket
- win-back eligibility
- category affinity
- offer sensitivity
- preferred channel
- fatigue score
- sentiment / experience risk
- VIP flag
- event responsiveness

Business feature examples

- segment revenue trend
- category revenue anomaly
- channel performance trend
- campaign overlap rate
- discount dependency trend
- seasonal demand index
- event demand uplift

This layer is the foundation for all agents.

Layer 3: Signal layer

The feature layer produces decision-ready signals.

Example signals

- `spend_decline_flag`
- `pre_churn_flag`

- `dormant_60_120_flag`
- `lost_120_plus_flag`
- `high_value_watch_flag`
- `cross_sell_ready_flag`
- `offer_optimization_needed_flag`
- `campaign_fatigue_flag`
- `negative_experience_flag`
- `segment_revenue_drop_flag`
- `seasonal_intent_spike_flag`

Signals are cheaper to consume than raw data.

Layer 4: Trigger and eligibility engine

This layer determines who actually needs processing.

Instead of scoring everyone every time, it asks:

- did something materially change?
- did a threshold get crossed?
- did a milestone occur?
- did a business anomaly appear?

Example triggers

- customer inactive for 30 / 60 / 120 days
- spend decline exceeds 20%
- delivery failure occurred
- support complaint logged
- campaign proposed for launch
- category revenue dropped beyond baseline
- seasonal demand spike detected
- new order changes category affinity

Only records that meet triggers move forward.

Layer 5: Supervisor agent layer

Supervisors own business domains and route work to child agents.

Supervisors do not deeply analyze all customers.
They first perform lightweight routing logic, then invoke only relevant child agents.

Layer 6: Child agent layer

Child agents execute only on:

- shortlisted customers
- shortlisted campaigns
- shortlisted segments
- shortlisted categories

They consume feature tables and signals, not raw source data.

Layer 7: Next Best Action layer

This is the master decision layer.

It does not rescan all data.
It consumes already prepared outputs from supervisors and child agents and decides:

- what to do
- when to do it
- through which channel
- whether to suppress all actions

Layer 8: Human approval and execution

Every final recommendation goes to:

- user review
- approval / modification / rejection
- downstream activation

This stays aligned with your human-in-the-loop model .

---

4. Supervisor and child execution design

A. Revenue Recovery Supervisor

Purpose

Recover revenue from declining, inactive, or lost customers.

Routing logic

- active but declining → Churn Prevention Agent
- inactive 60–120 days → Dormant Customer Activation Agent
- inactive 120+ days → Win-Back Strategy Agent

Child agents

- Churn Prevention Agent
- Dormant Customer Activation Agent
- Win-Back Strategy Agent

Compute rule

Only one child agent should run per customer in most cases.

B. Customer Value Growth Supervisor

Purpose

Grow value from existing customers.

Routing logic

- mid-tier with upside → Loyalty Growth Agent
- ready for adjacent category / premium move → Cross-Sell & Upsell Intelligence Agent
- investment allocation decision needed → Customer Lifetime Value Optimization Agent

Child agents

- Loyalty Growth Agent
- Cross-Sell & Upsell Intelligence Agent
- Customer Lifetime Value Optimization Agent

Compute rule

CLV and growth opportunity signals should be reused by all three, not recalculated separately.

C. Campaign Intelligence Supervisor

Purpose

Validate and optimize campaign plans before launch.

Routing logic

- first predict performance
- if viable, optimize incentives
- before approval, run risk / fatigue guardrails

Child agents

- Campaign Outcome Prediction Agent
- Offer Optimization Agent
- Campaign Risk & Fatigue Agent

Compute rule

Campaigns should be evaluated at campaign/audience level first, not customer-by-customer unless necessary.

D. Customer Experience Guardian Supervisor

Purpose

Protect customers from experience-driven attrition.

Routing logic

- negative experience detected → Customer Sentiment & Experience Agent
- same customer is high-value / VIP → High-Value Customer Protection Agent escalation

Child agents

- Customer Sentiment & Experience Agent
- High-Value Customer Protection Agent

Compute rule

This should be highly event-driven, not periodic full scans.

E. Revenue Intelligence Supervisor

Purpose

Detect emerging revenue loss and understand where it is happening.

Routing logic

- decline detected in category / segment / cohort → Revenue Loss Detection Agent
- if pattern unclear or new behavior emerges → Segment Discovery Agent

Child agents

- Revenue Loss Detection Agent
- Segment Discovery Agent

Compute rule

Primarily segment-level and cohort-level processing, not full customer-level rescoring.

F. Seasonal Demand Intelligence Supervisor

Purpose

Improve event and seasonal planning.

Routing logic

- seasonal/event window approaching → Seasonal & Event Intelligence Agent
- if new behavior cluster appears → Segment Discovery Agent support

Child agents

- Seasonal & Event Intelligence Agent
- Segment Discovery Agent

Compute rule

Mostly weekly or periodic, with event-specific intensification.

G. Master Orchestrator

Next Best Action Agent

Purpose

Choose the single best action or no action.

Input sources

- Revenue Recovery Supervisor output
- Customer Value Growth Supervisor output
- Campaign Intelligence Supervisor output
- Experience Guardian Supervisor output
- Revenue Intelligence Supervisor output
- Seasonal Intelligence Supervisor output

Compute rule

Should consume prepared recommendations and suppression rules only.

---

5. Processing strategy by compute cost

Tier 1: Cheap rules

Use on large populations.

Examples:

- days since last purchase
- spend drop threshold
- campaign count in last 14 days
- top 10% LTV flag
- complaint count
- event responsiveness flag

Purpose:

- reduce the candidate pool

Tier 2: Medium-cost scoring

Use only on eligible populations.

Examples:

- churn score
- reactivation probability
- win-back feasibility
- offer sensitivity

- CLV opportunity score
- fatigue risk score

Purpose:

- rank and prioritize

Tier 3: Expensive decisioning / reasoning

Use only on top candidates.

Examples:

- tailored intervention recommendation
- departure reason diagnosis
- campaign scenario simulation
- final next-best-action resolution

Purpose:

- produce actionable recommendations

---

6. Invocation policy

General rules

1. No child agent should scan raw enterprise tables directly.
2. All child agents must consume precomputed features/signals.
3. A child agent should run only if trigger conditions are satisfied.
4. Where child agents are mutually exclusive, only one should run.
5. Expensive reasoning should happen only after eligibility filtering.
6. The NBA agent should combine outputs, not restart analysis.

---

7. Processing cadence

Real-time / near real-time

Use for urgent or event-based actions.

- delivery failures
- complaints / tickets

- cart abandonment
- VIP customer deviation
- active campaign suppression decisions

Daily

Use for operational customer intelligence.

- spend decline
- churn screening
- dormant screening
- cross-sell readiness
- fatigue updates
- category affinity refresh

Weekly

Use for deeper optimization.

- CLV optimization refresh
- segment discovery
- offer sensitivity recalibration
- seasonal/event demand shifts

Monthly

Use for strategic recalibration.

- model retraining
- threshold updates
- policy tuning
- long-term segment evolution

---

8. Shared feature store design

Customer feature store

One row per customer, updated incrementally.

Suggested columns:

- customer_id

- last_purchase_date
- days_since_last_purchase
- purchase_frequency_30d
- purchase_frequency_90d
- avg_order_value
- spend_velocity_30d
- spend_decline_pct
- ltv_current
- ltv_projected
- churn_score
- dormant_bucket
- winback_eligibility_score
- category_affinity_vector
- offer_sensitivity_score
- fatigue_score
- preferred_channel
- sentiment_risk_score
- vip_flag
- event_affinity_score
- cooldown_until

Campaign feature store

One row per campaign/audience version.

Suggested columns:

- campaign_id
- target_audience_size
- audience_overlap_7d
- fatigue_risk_score
- predicted_open_rate
- predicted_click_rate
- predicted_conversion_rate
- predicted_revenue

- discount_dependency_risk
- recommended_offer_type
- recommended_offer_value

Segment / category feature store

Suggested columns:

- segment_id / category_id
- revenue_baseline
- revenue_current
- anomaly_score
- seasonal_index
- demand_spike_flag
- engagement_shift_score
- campaign_saturation_score

---

9. Event-driven optimization model

Recommended event types

- OrderPlaced
- SupportComplaintLogged
- ProductViewed
- CartAbandoned
- CampaignPlanned
- CampaignSent
- LoyaltyTierChanged
- InactivityMilestoneReached
- SegmentRevenueDropDetected
- SeasonalSignalDetected

Each event should update features incrementally instead of causing a full platform rescan.

---

10. Example execution flow

Example 1: Revenue recovery

1. Daily feature update computes spend decline and inactivity windows.
2. Trigger engine finds customers with:
   - 20%+ spend decline, or
   - 60+ days inactivity, or
   - 120+ days inactivity
3. Revenue Recovery Supervisor routes:
   - decline → Churn Prevention
   - 60–120 → Dormant Activation
   - 120+ → Win-Back
4. Each child agent produces recommendation only for assigned customers.
5. NBA decides whether to trigger recovery, cross-sell, or no action.

Example 2: Campaign planning

1. Marketer drafts campaign.
2. Campaign Intelligence Supervisor starts processing.
3. Outcome Prediction Agent estimates likely performance.
4. Offer Optimization Agent recommends best incentive structure.
5. Risk & Fatigue Agent validates overlap, fatigue, and discount dependency.
6. Final optimized campaign goes to NBA or approval workflow.

Example 3: Experience recovery

1. Delivery delay event lands in platform.
2. Customer experience features update immediately.
3. Trigger engine checks severity + customer value.
4. Experience Supervisor routes:
   - standard customer → Sentiment & Experience Agent
   - VIP customer → High-Value Protection escalation
5. Recovery recommendation goes to approval.

---

11. Compute optimization rules for design doc

You can use these as formal architecture principles:

Compute Optimization Principles

- Shared features must be computed once and reused across all agents.
- Agents must consume feature tables and signal tables rather than raw operational tables.
- Full-population rescoring should be avoided except during controlled recalibration or retraining cycles.
- Child agents must only execute on eligible populations identified by the trigger engine.
- Supervisor agents must apply lightweight routing before invoking any expensive child process.
- Event-driven incremental updates should be preferred over repeated historical rescans.
- Next Best Action must operate as a decision combiner, not as a duplicate analytics engine.
- Processing cadence must align to business urgency and signal volatility.

---

12. Recommended architecture summary table

Layer	Purpose	Compute Style	Output
Source Data	Capture raw enterprise data	Ingestion only	Raw records
Feature Layer	Compute reusable metrics once	Batch + incremental	Feature tables
Signal Layer	Convert features into business signals	Lightweight	Eligibility flags
Trigger Engine	Identify records needing action	Rule-based	Candidate sets
Supervisor Layer	Route to right decision domain	Lightweight orchestration	Routed workloads
Child Agent Layer	Perform focused analysis	Scoped compute	Recommendations

NBA Layer	Resolve conflicts and choose action	Decision orchestration	Final action
Human Approval	Review and approve	Manual	Approved execution

---

13. Final recommended operating model

Best-fit Custonomy model

Custonomy should run as a **signal-driven, supervisor-orchestrated, feature-store-backed decision platform**.

That means:

- **data scanned once**
- **signals reused everywhere**
- **supervisors control invocation**
- **child agents run only when justified**
- **NBA chooses the final action**
- **human approves execution**

This will keep Custonomy:

- scalable
- explainable
- fast
- cost-efficient
- enterprise-ready